SysMonitor AI

Real-Time System Performance Monitor

Technical Documentation & Project Overview

Course	Operating Systems & Computer Graphics
Stack	Python · FastAPI · React · TypeScript · Three.js
Platform	Windows & Red Hat Enterprise Linux 10
AI Model	Isolation Forest (scikit-learn)
Protocol	WebSockets (real-time)

1. Project Overview

SysMonitor AI is a full-stack, real-time system performance monitoring application that combines operating system-level data collection, machine learning-based anomaly detection, and interactive 3D data visualization. The project was built as a practical demonstration of both OS concepts (process management, resource monitoring, IPC) and Computer Graphics principles (3D rendering, shaders, real-time animation).

The application runs a Python backend that continuously samples system metrics, feeds them into a trained AI model to detect unusual behavior, and streams everything in real-time to a React frontend where the data is displayed as interactive 3D charts and an AI alert panel.

2. System Architecture

Layer	Technology	Responsibility
Data Collection	psutil (Python)	Reads CPU, RAM, Disk, Network, Processes from OS
AI Engine	scikit-learn Isolation Forest	Detects anomalies in real-time metric streams
Backend API	FastAPI + Uvicorn	REST endpoints + WebSocket server
Real-time Transport	WebSockets	Pushes live data from backend to frontend every 2s
Frontend Framework	React 18 + TypeScript	Component-based UI, state management, hooks
3D Visualization	Three.js + React Three Fiber	Interactive 3D bar charts with OrbitControls
Post-Processing	@react-three/postprocessing	Bloom glow effect on high-value bars
Build Tool	Vite	Fast dev server and production bundler

Package Manager	pip (backend) / npm (frontend)	Dependency management for both tiers

3. Backend — Python / FastAPI

3.1 Data Collection (collector.py)

The MetricsCollector class uses the psutil library to read system performance data directly from the operating system. On Linux, psutil reads from kernel interfaces like /proc/stat, /proc/meminfo, /proc/net/dev, and /proc/diskstats — making it a direct demonstration of OS concepts covered in the course.

Metric	psutil Call	OS Source (Linux)
CPU % per core	`cpu_percent(percpu=True)`	/proc/stat
RAM usage	`virtual_memory()`	/proc/meminfo
Disk I/O	`disk_io_counters()`	/proc/diskstats
Network I/O	`net_io_counters()`	/proc/net/dev
Process list	`process_iter()`	/proc/[pid]/stat

3.2 API Server (main.py)

FastAPI provides the backend server with two types of endpoints:

- WebSocket /ws — streams live snapshots + AI alerts to the frontend every 2 seconds
- POST /alerts/{id}/acknowledge — marks an alert as seen by the user
- GET /alerts — returns filtered alert history with optional severity filter
- GET /health — returns AI training status and buffer size

3.3 Python Libraries Used

Library	Version	Purpose
`fastapi`	0.111.0	Modern async web framework for REST + WebSocket
`uvicorn`	0.29.0	ASGI server to run FastAPI
`psutil`	5.9.8	Cross-platform system metrics collection
`scikit-learn`	1.4.2	Isolation Forest anomaly detection model
`numpy`	1.26.4	Numerical arrays for feature vectors and training data
`pandas`	2.2.2	Data manipulation and time-series handling
`pydantic`	built-in	Data validation and serialization of metrics models
`pytest`	8.2.0	Unit and integration testing framework

4. AI Anomaly Detection — Isolation Forest

4.1 Why Isolation Forest?

Isolation Forest is an unsupervised machine learning algorithm specifically designed for anomaly detection. It works by randomly partitioning data — anomalies, being rare and different, require fewer partitions to isolate than normal data points. This makes it extremely efficient for real-time streams where we have no labeled 'anomaly' examples to train on.

Key advantages for this use case: no labeled training data needed, works well on high-dimensional data (our 9-feature vector), handles concept drift with periodic retraining, and runs fast enough for real-time inference every 2 seconds.

4.2 Feature Vector

Each system snapshot is converted into a 9-dimensional feature vector:

Index	Feature	Source Metric
0	`cpu_overall`	Overall CPU utilization %
1	`ram_percent`	RAM usage %
2	`disk_read_mbps`	Disk read speed MB/s
3	`disk_write_mbps`	Disk write speed MB/s
4	`net_upload_mbps`	Network upload speed MB/s
5	`net_download_mbps`	Network download speed MB/s
6	`top_proc_cpu_max`	Highest CPU% among top processes
7	`top_proc_mem_max`	Highest RAM% among top processes
8	`process_count`	Number of top processes tracked

4.3 Training Lifecycle

- Startup: model is untrained, no alerts are generated
- Buffer fills: every snapshot's features are stored in a rolling deque (max 500)
- First train: once 50 samples collected, Isolation Forest is fit on the buffer
- Retraining: model retrains every 50 new samples to adapt to usage pattern changes
- Detection: each new snapshot is scored — score of -1 means anomaly detected

4.4 Severity Classification

Severity	Z-Score Range	Color	Meaning
Low	2.5 — 3.0	Blue	Slight deviation from normal baseline
Medium	3.0 — 4.0	Orange	Notable anomaly worth attention

High	4.0 — 5.0	Red-Orange	Significant system stress detected
Critical	> 5.0	Bright Red	Extreme deviation — immediate attention

5. Frontend — React / TypeScript / Three.js

5.1 Frontend Libraries Used

Library	Version	Purpose
`react`	18.3.0	Component-based UI framework
`typescript`	5.4.5	Type-safe JavaScript for all components
`three.js`	0.165.0	Core WebGL 3D rendering engine
`@react-three/fiber`	8.16.8	React renderer for Three.js scenes
`@react-three/drei`	9.105.4	Helpers: OrbitControls, Text, Grid
`@react-three/postprocessing`	latest	Bloom glow shader post-processing
`vite`	5.2.11	Fast build tool and dev server
`lucide-react`	0.379.0	Icon library for UI elements
`date-fns`	3.6.0	Timestamp formatting for alert cards

5.2 Component Structure

Component	File	Purpose
App	`App.tsx`	Root layout, WebSocket connection, state orchestration
StatusBar	`StatusBar.tsx`	Connection status, AI training state, live clock
MetricGauge	`MetricGauge.tsx`	SVG circular gauge for CPU/RAM/Disk/Network
Chart3D	`Chart3D.tsx`	Interactive 3D bar chart using React Three Fiber
AlertPanel	`AlertPanel.tsx`	Scrollable alert list with severity filters
AlertCard	`AlertCard.tsx`	Individual alert with severity badge + acknowledge

6. 3D Charts — Computer Graphics Implementation

The 3D visualization component demonstrates core Computer Graphics principles using Three.js and WebGL under the hood via React Three Fiber.

6.1 Rendering Pipeline

- Scene Graph: each bar is a Three.js Mesh with BoxGeometry and MeshStandardMaterial
- Camera: PerspectiveCamera at position [0, 8, 16] with 50 degree field of view

- Lighting: AmbientLight + DirectionalLight with shadow casting + PointLight for fill
- Materials: metalness=0.3, roughness=0.4 for physically-based rendering (PBR)
- Post-processing: Bloom effect via EffectComposer adds glow to high-value bars

6.2 Interactivity (Course Requirement)

OrbitControls from @react-three/drei provides full 3D interaction: left-click drag to rotate the chart in any direction, scroll wheel to zoom in/out, and right-click drag to pan. Damping factor of 0.05 creates smooth inertial movement.

6.3 Real-Time Animation

The useFrame hook runs inside the WebGL render loop (60fps). Bar heights are lerped (linearly interpolated) toward target values at factor 0.08 per frame, creating smooth transitions as new data arrives every 2 seconds. Color changes dynamically: green below 65% threshold, orange 65-85%, red above 85%.

7. Real-Time Communication — WebSockets

WebSockets provide a persistent, full-duplex connection between the backend and frontend. Unlike HTTP polling, the server pushes data to the client the moment it's ready, with no request overhead. This is essential for real-time monitoring at 2-second intervals.

- Connection: ws://localhost:8000/ws
- Payload: JSON with type, snapshot data, and new alerts array
- Reconnection: exponential backoff starting at 1s, doubling to max 30s
- On connect: last 50 alerts sent immediately to populate UI

8. Operating Systems Course Relevance

This project directly applies core OS concepts:

- Process Management: reads all running processes, their PID, CPU and memory usage via /proc
- Resource Monitoring: tracks CPU scheduler data, memory allocator stats, and I/O subsystem counters
- Inter-Process Communication: WebSocket server demonstrates IPC between backend and frontend processes
- File System: disk I/O monitoring reads kernel block device statistics
- Scheduling: per-core CPU tracking shows how the OS distributes work across cores
- Virtual Memory: RAM metrics distinguish between physical usage and available pages

9. Running the Project

Prerequisites: Python 3.11+, Node.js 20+, npm 10+

Clone and run:

```
git clone https://github.com/your-repo/sysmonitor.git
cd sysmonitor/sysmon
chmod +x start.sh && ./start.sh
```

Access points after startup:

- Frontend UI: http://localhost:5173
- Backend API: http://localhost:8000
- API Docs: http://localhost:8000/docs

Run tests:

```
cd backend && pytest tests/ -v
```